

课程设计报告

自走棋对战系统

C++17 + SFML 2.6.2

课程名称

计算机程序设计基础 2

课题名称

自走棋对战系统

班 级

学 号

姓 名

指导教师

2026 年 8 月 22 日

摘要

本项目面向《计算机程序设计基础 2》课程设计要求，使用 C++17 与 SFML 2.6.2 独立实现一个可配置、可测试、可发布的自走棋对战系统。系统提供主菜单、帮助、地图选择、分队选择、AI 策略选择、商店购买、刷新、拖拽部署、合成、出售、复活、提前开战、固定步长战斗、主动技能、暂停继续、回合结算与结果页面。核心规则通过 AutoChessCore 静态库与 SFML 表示层隔离，真人和 AI 均通过统一 GameCommand 接口提交操作，界面与 AI 仅获取 ReadOnlyGameView 值拷贝快照。数据由六个 UTF-8 文本文件描述，包含两张地图、三个分队、五种单位和五种技能。项目采用抽象类、虚函数多态、多重继承与运算符重载，并建立独立测试程序。最终 Release x64 测试统计为 402 项通过、0 项失败；18 组 AI 组合均在 3 回合内完成且命令失败数为 0；独立 dist 目录在项目外工作目录可启动。

关键词：自走棋；C++17；SFML；面向对象；策略模式；固定步长；自动测试

目录

摘 要	2
目 录	2
1 系统需求分析	4
1.1 设计目的与意义	4
1.2 功能需求	4
1.3 非功能需求	4
1.4 输入与输出要求	5
2 系统总体设计	7
2.1 总体架构	7
2.2 关键设计原则	7
2.3 模块依赖与数据流	8
3 系统详细设计	8
3.1 数据文件设计	8
3.2 类层次、抽象类与多态	8
3.3 运算符重载	9
3.4 经济与准备阶段	10
3.5 战斗、技能与结算	10

3.6 AI 设计	10
3.7 界面设计	10
4 系统实现	10
4.1 固定步长主循环	10
4.2 多重继承与单位角色	11
4.3 统一命令	11
4.4 独立发布路径	11
5 系统测试	12
5.1 测试环境与方法	12
5.2 最终自动测试结果	12
5.3 启动故障与运行验收	12
5.4 独立发布目录	13
6 运行界面与结果	13
7 系统调试	19
7.1 CMake 工程文件名错误	19
7.2 发布包数据路径依赖开发目录	19
7.3 同帧索敌并列规则缺少直接断言	19
7.4 调试方法总结	20
8 使用说明与编程体会	20
8.1 发布程序使用步骤	20
8.2 构建与测试步骤	20
8.3 编程体会	20
9 总结与展望	21
附录 A 关键源程序与源程序清单	22
附录 B 完整课程设计任务书与评分表	24

1 系统需求分析

1.1 设计目的与意义

自走棋的核心难点不在单一动画，而在多阶段状态、经济规则、单位持久状态、固定步长战斗和 AI 合法决策之间的一致性。本项目以课程要求的购买、升级、排列和对战为基础，进一步采用数据驱动、统一命令和自动测试，使规则变更能够通过配置完成，使真人、AI、测试脚本共用同一套核心入口，从而训练面向对象设计、模块边界划分、可测试性和发布工程能力。

项目的个性化设计体现在三点：第一，五类单位和五种技能完全由配置定义数值；第二，三种 AI 在不读取对手私有信息的前提下使用同一只读快照接口；第三，Release 程序优先读取可执行文件旁的 data 目录，可脱离源码与构建目录独立运行。

1.2 功能需求

功能模块	需求	验收边界
启动与菜单	检查配置、地图、中文字体和 SFML DLL；提供开始、帮助和退出。	缺失资源时明确报错，不进入不完整主循环。
开局选择	选择 2 张地图之一、3 个玩家分队之一和 3 种 AI 策略之一。	选择顺序固定，非法阶段命令被核心拒绝。
准备与经济	收入、败者补助、6 槽商店、刷新、购买、8 槽备用区、部署、合成、出售、复活。	金币、容量、部署区和格子占用均原子校验。
战斗	单位移动、索敌、普攻、普通治疗、主动技能、守卫伤害和 60 秒超时。	固定 1/60 秒步长；同帧事件可重复。
回合与结果	最多 3 回合；结算死亡、存活、到达守卫和超时状态。	守卫归零立即结束；最大回合后比较守卫。
暂停与重开	暂停冻结核心时间，继续恢复；结果页支持重开、再来一局、返回菜单。	暂停期间仍响应窗口关闭和界面绘制。
AI	进攻型、防守型、路线型策略适配任意正式地图与分队。	只生成统一命令，不越权修改 Match。
测试与发布	独立测试程序、非法输入样本、日志、dist 目录。	Release 402 PASS / 0 FAIL，dist 文件清单固定。

1.3 非功能需求

- 可编译：使用 CMake Presets、Ninja、MSVC x64 和 C++17；核心库不依赖 SFML。
- 可运行：Release 程序在 1280×720 窗口中稳定运行，退出码可区分成功与启动失败。
- 可验证：核心规则不依赖真实时间和窗口，可由 AutoChessTests.exe 直接执行。
- 可追溯：最终提交号、随机种子、测试日志、发布文件和非法输入样本均记录 SHA-256。
- 可发布：dist 只包含 1 个 EXE、3 个 SFML Release DLL 和 6 个数据文件。
- 可维护：规则数值集中在 data 文本文件；界面只生成命令并显示结果。

1.4 输入与输出要求

类型	输入/输出	具体内容
用户输入	鼠标	按钮点击、商店购买、拖拽部署/回备用区/出售区、战斗单位选择、技能释放、暂停和继续。
文件输入	UTF-8 文本	game.cfg、units.cfg、skills.cfg、factions.cfg、map_01.map、map_02.map。
程序输出	SFML 窗口	菜单、帮助、地图、HUD、商店、单位、血条、技力、路线、提示、暂停层和结果页。
诊断输出	标准错误/退出码	配置文件路径与行号、字体候选路径、缺 DLL 系统退出码。
测试输出	日志	PASS/FAIL、回合摘要、18 组 AI MATRIX、文件哈希和构建环境。

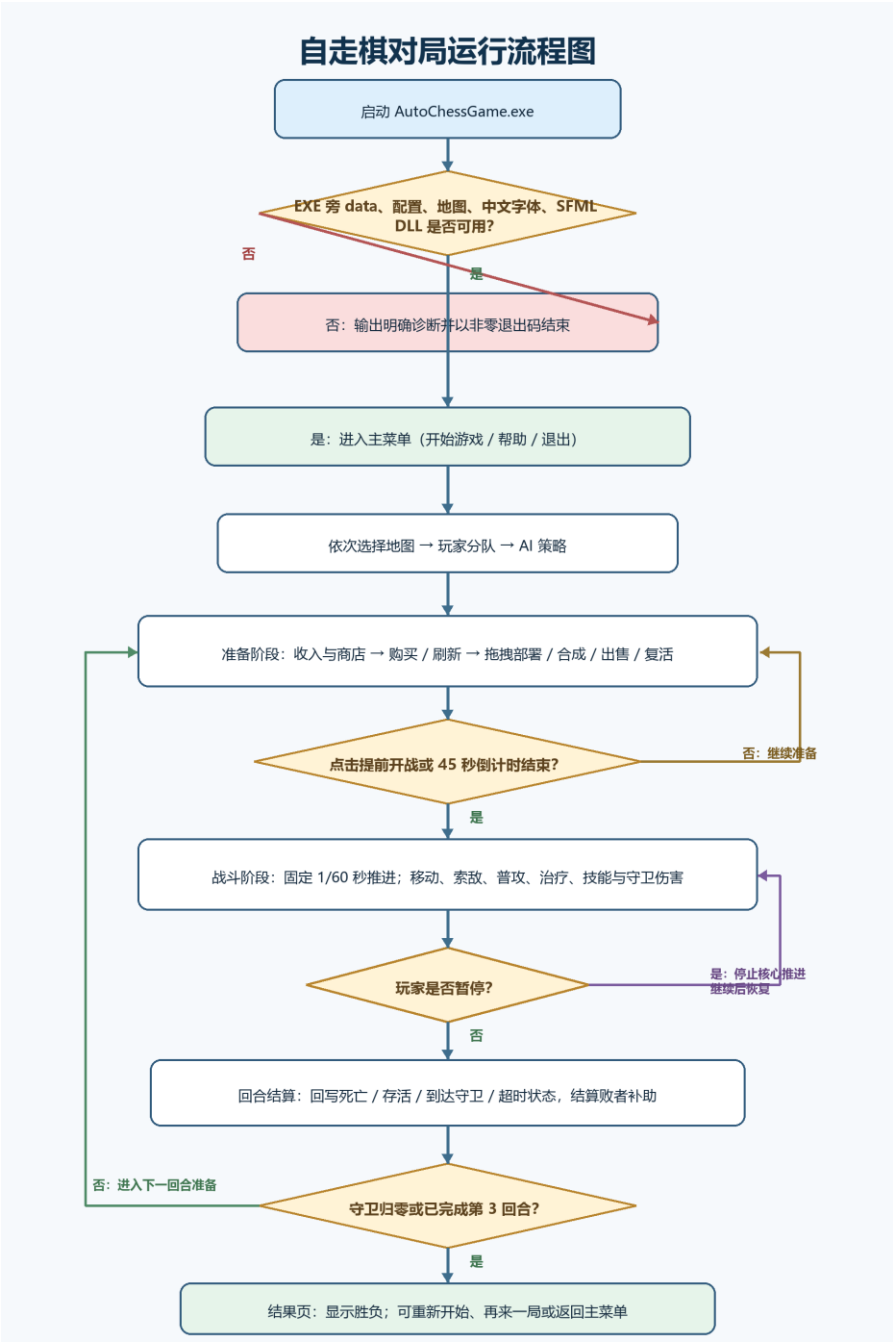


图 1-1 自走棋对局运行流程图

2 系统总体设计

2.1 总体架构

工程由 AutoChessCore、AutoChessGame、AutoChessTests 和 AutoChessDist 四个 CMake 目标组成。AutoChessCore 保存配置、模型、经济、战斗、技能、对局和 AI，不包含 SFML 头文件；AutoChessGame 负责窗口、页面、渲染、鼠标事件和运行路径；AutoChessTests 直接链接核心库并复用运行路径工具；AutoChessDist 在构建后组装固定发布目录。

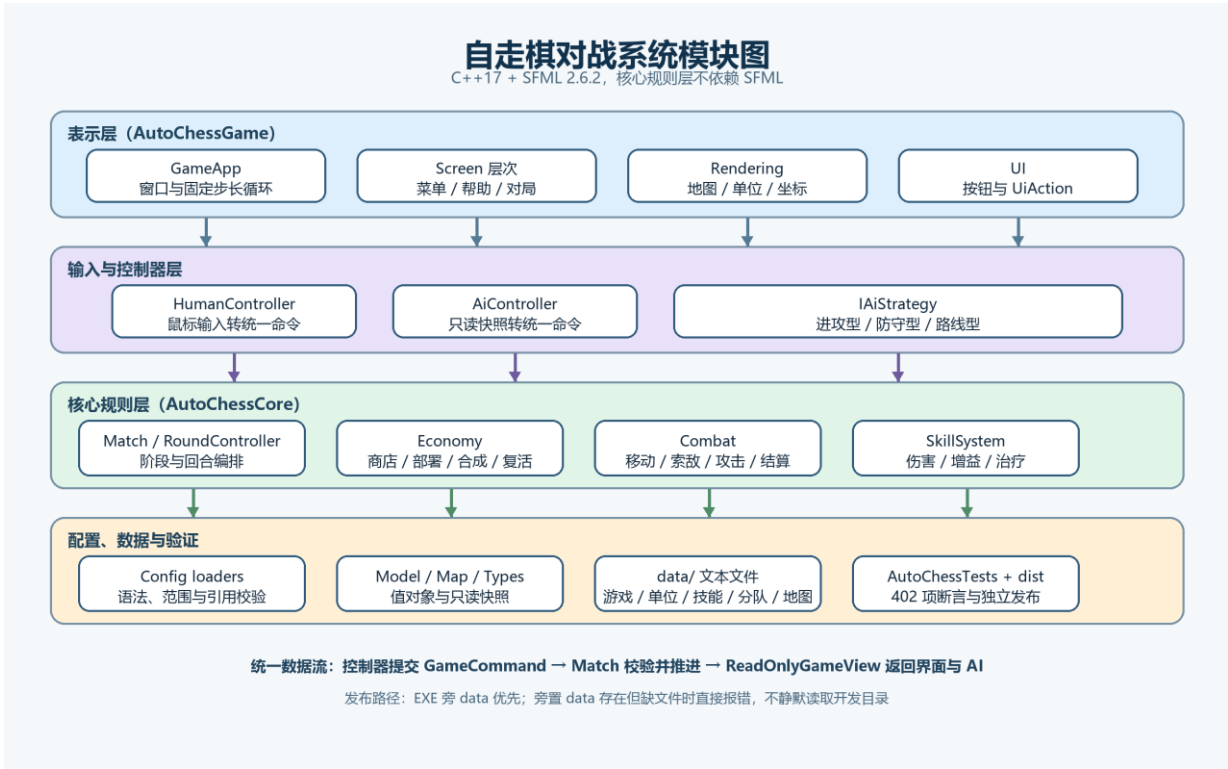


图 2-1 系统功能模块图

2.2 关键设计原则

原则	实现方式与收益
持久单位与战斗单位分离	OwnedUnit 保存跨回合状态, BattleUnit 只服务当前战斗; 避免死亡、复活与满血恢复相互污染。
统一命令入口	HumanController、AiController 和脚本控制器均提交 GameCommand; Match 负责阶段、归属、资源与规则校验。
只读快照	ReadOnlyGameView 是值拷贝, 不暴露 Match 内部引用; AI 看不到对手金币、商店和死亡列表。
固定步长	准备与战斗均按 1/60 秒推进, 随机种子固定为 20260814, 结果可以复现。
数据驱动	单位、技能、分队、经济和地图数值不硬编码在界面; 配置错误带文件与行号。
发布优先路径	EXE 旁 data 存在时必须使用它; 只有旁置 data 目录不存在时才允许开发目录回退。

2.3 模块依赖与数据流

1. GameApp 从 main.cpp 接收已经选择的数据目录，加载 ConfigBundle 和中文字体。
2. Screen 将鼠标操作转换为 UiAction，HumanController 再转换为 GameCommand。
3. AiController 读取 B 方 ReadOnlyGameView，并委托 IAiStrategy 选择至多一条合法命令。
4. Match::submit 校验并执行命令，Match::step 推进准备倒计时、BattleSimulation 或回合结算。
5. GameApp 将新的 ReadOnlyGameView 交给 MatchScreen；Renderer 只负责像素坐标和绘制。
6. AutoChessTests 直接调用核心对象，以相同配置和种子验证单元、集成、AI 与发布路径。

3 系统详细设计

3.1 数据文件设计

所有正式数据使用 UTF-8 无 BOM 文本。加载顺序固定为 game、skills、units、factions、map_01、map_02，以保证技能先于单位、单位先于分队引用。解析器拒绝重复节、重复字段、未知字段、非法整数/比例、缺失字段、范围越界和悬空引用。

文件	主要内容	最终规模
game.cfg	回合、时间、经济、容量、比例与随机种子	3 回合；45 秒准备；60 秒战斗；种子 20260814
units.cfg	生命、攻防、射程、速度、价格、技力、普通行为	铁卫、决斗者、游侠、奥术师、医师
skills.cfg	目标规则、效果、范围、等级值、持续时间和增益方式	铁壁姿态、破阵斩、集中射击、奥术爆裂、战地治疗
factions.cfg	守卫、部署上限、价格倍率和单位修正	坚壁、突击、行军 3 个分队
map_01.map	11×7 对称双路训练场	A/B 各 2 条正交路线
map_02.map	15×9 多路线峡谷	A/B 各 3 条短/中/长路线

3.2 类层次、抽象类与多态

项目定义多个抽象层次。Unit、Skill、Screen 为抽象基类；ISkillUser、IAiStrategy、IPlayerController 为纯接口。ActiveSkillUnit 同时继承 Unit 和 ISkillUser，满足多重继承要求；五个具体单位只固定角色，数值仍来自配置。三种 AI 策略通过 IAiStrategy 的虚函数实现不同购买、部署和技能选择偏好。

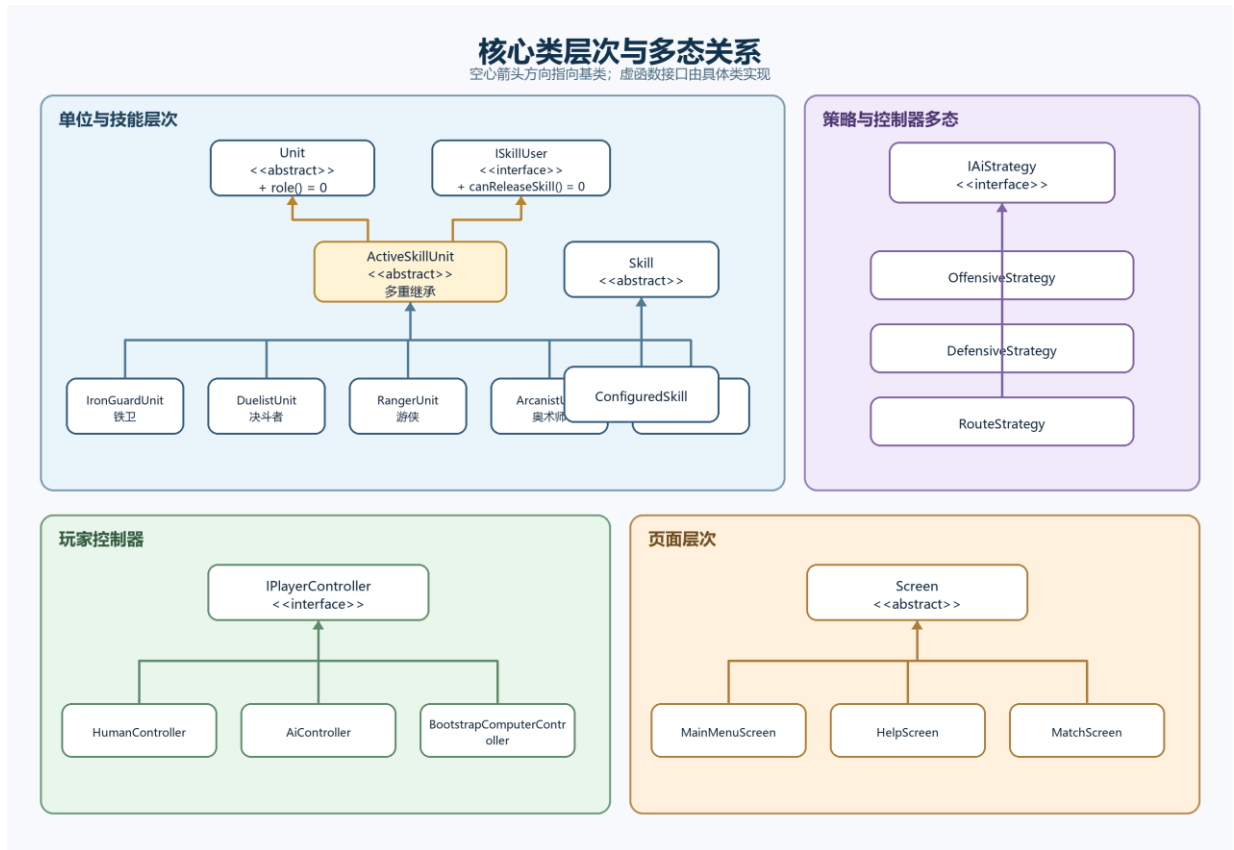


图 3-1 核心类层次与多态关系

代表性类	主要属性（均多于 3 项）	主要方法（均多于 3 项）
GameApp	dataDirectory_、window_、font_、config_、match_、humanController_、computerController_、screen_、frameClock_ 等	run、loadConfiguration、loadChineseFont、handleEvent、processUiAction、startNewGame、fixedUpdate、render 等
Match	config_、randomEngine_、phase_、currentRound_、地图/分队/AI 选择、双方玩家与商店、battle_ 等	submit、step、viewFor、selectMap、selectFaction、startCombat、releaseSkill、settleCurrentRound 等
BattleSimulation	units_、map_、skills_、timeoutFrames_、currentFrame_、finished_、守卫伤害和 summary_	step、releaseSkill、moveUnit、updateTargets、applyBasicActions、applyFrameGuardDamage、finish 等
MatchScreen	只读快照、选择项、商店卡、复活卡、拖拽状态、坐标换算、按钮、HUD、暂停和技能状态	handleEvent、update、draw、setView、rebuildChoices、finishBasicDrag、drawCombatUnits、refreshCombatWidgets 等

3.3 运算符重载

为保持值对象比较和日志输出简洁，GridPosition 重载 ==、!=、<，BattlePosition 和 UnitIdentity 重载 ==，RoundSummary 与 MatchResultSummary 重载流输出 operator<<。这些重载分别服务地图坐标集合、测试断言和可追溯日志，不改变业务规则。

3.4 经济与准备阶段

每回合开始按配置发放 5 金币，败者额外获得 2 金币。商店有 6 个槽位，刷新费为 2；备用区容量为 8。购买、刷新、部署、回备用区、合成、出售和复活均先验证再一次性修改状态。两个同类型同等级单位可合成，最高 3 级；合成返还 40%，出售返还 75%，复活成本为原价的 50%，统一使用冻结的四舍五入规则。

3.5 战斗、技能与结算

BattleSimulation 每帧依次处理无效目标清理、移动、索敌、普通行为、技能状态和守卫伤害。同一帧的伤害先计算后统一生效，使双方互相击杀和守卫同时归零可重复。普通攻击物理伤害具有最低 5 点，法术伤害经过法抗边界，攻击速度支持 0 与 600。医师普通行为为治疗，目标选择过滤满血单位并限制最大生命。

回合结束后，RoundController 把战斗结果按单位 ID 回写：死亡进入死亡列表，存活、到达守卫和超时单位回到活动名单并恢复下一回合状态。守卫归零立即判负；完成第 3 回合后比较守卫，数值相同则平局。

3.6 AI 设计

策略	决策偏好	可观察差异
进攻型 Offensive	优先高攻击/高输出购买与前置部署，积极释放伤害技能。	快速形成伤害压力。
防守型 Defensive	优先生存、治疗与保护部署，技能选择偏向防御和恢复。	提高阵容稳定性。
路线型 Route	评估路线长度、部署格和价格效率，优先路线收益。	适配两张不同路线地图。

3.7 界面设计

SFML 层使用 1280×720 窗口。BoardTransform 集中完成格子与像素坐标换算；MapRenderer 绘制障碍、部署区、守卫和可切换路线；UnitRenderer 绘制单位、生命、技力和目标线。Screen 层只产生 UiAction，MatchScreen 只依据 ReadOnlyGameView 刷新控件，不直接写核心状态。

4 系统实现

4.1 固定步长主循环

GameApp 把真实帧间隔限制在最大值内，再用累加器按 FixedStepSeconds 重复调用 fixedUpdate。渲染频率和模拟步长分离后，暂停、测试和不同机器帧率不会改变规则顺序。

```
// 此代码块在窗口存活期间限制累计时间并按固定步长更新核心。
while (window_.isOpen())
{
    sf::Event event{};
    while (window_.pollEvent(event))
    {
        handleEvent(event);
    }

    const float frameSeconds = std::min(
        frameClock_.restart().asSeconds(), MaximumFrameSeconds);
    accumulatorSeconds_ += frameSeconds;
    while (accumulatorSeconds_ >= FixedStepSeconds)
    {
        fixedUpdate();
        accumulatorSeconds_ -= FixedStepSeconds;
    }

    updateMatchScreen();
    render();
}
```

4.2 多重继承与单位角色

```
// 此抽象基类保存配置定义并要求具体单位声明自己的角色。
class Unit
{
public:
    virtual ~Unit() = default;
    virtual UnitRole role() const noexcept = 0;
};

// 此抽象类通过明确的多重继承为五个主动技能单位复用通用规则。
class ActiveSkillUnit : public Unit, public ISkillUser
{
public:
    const std::string& skillId() const noexcept override;
    bool canReleaseSkill(int currentMana, int maxMana, bool skillActive) const noexcept override;
};
```

4.3 统一命令

```
// 此变体限定统一入口能够接收的全部命令类型。
using GameCommandPayload = std::variant<
    SelectMapCommand, SelectFactionCommand, SelectAiStrategyCommand,
    RefreshShopCommand, PurchaseUnitCommand,
    MoveToDeploymentCommand, MoveToReserveCommand,
    MergeUnitsCommand, SellUnitCommand, ReviveUnitCommand,
    StartCombatCommand, ReleaseSkillCommand>;

// 此结构把执行阵营和具体命令负载封装成统一输入。
struct GameCommand
{
    MapSide actor = MapSide::Unknown;
    GameCommandPayload payload = RefreshShopCommand{};
};
```

4.4 独立发布路径

程序不能依赖启动时的当前工作目录。入口通过 `GetModuleFileNameW` 得到 EXE 目录；只要旁边存在 `data` 目录，就固定使用该目录并暴露缺文件错误。这样既支持开发构建目录回退，也防止发布包缺文件时静默读取源码数据。

```
// 此函数让发布包使用自身 data，同时保留构建目录缺 data 时的开发回退。
std::filesystem::path selectDataDirectory()
```

```

const std::filesystem::path& executableDirectoryPath,
const std::filesystem::path& developmentDataDirectory)
{
    const std::filesystem::path adjacentDataDirectory =
        executableDirectoryPath / "data";
    std::error_code error;

    // 此代码块只要确认程序旁存在目录，就固定使用该目录并暴露其中缺文件错误。
    if (std::filesystem::is_directory(adjacentDataDirectory, error))
    {
        return adjacentDataDirectory;
    }
    return developmentDataDirectory;
}

```

5 系统测试

5.1 测试环境与方法

项目	最终环境/方法
操作系统与架构	Windows x64
语言与图形库	C++17, SFML 2.6.2 (项目已有依赖)
编译器	MSVC 19.51.36256.0
构建工具	CMake 3.29.2, Ninja 1.12.0, msvc-release 预设
自动测试	AutoChessTests.exe, 固定随机种子 20260814
人工验收	192 DPI (200% 缩放), 1280×720 客户区, 13 阶段 GUI 流程
发布验收	将 dist 复制到项目外目录, 以 D:\ 为工作目录启动

5.2 最终自动测试结果

测试范围	结果	证据
全部 Release 断言	402 PASS / 0 FAIL, 退出码 0	docs/test-results/final-release-tests-2026-08-22.log
最高优先级规则	配置、地图、伤害、索敌、治疗、经济、跨回合、胜负全部通过	priority-regression-matrix-2026-08-22.md
完整三回合脚本	3 条回合摘要与 1 条最终摘要	最终 Release 日志
AI 组合矩阵	3 策略 × 2 地图 × 3 分队 = 18 组	18 行 AI MATRIX
AI 命令合法性	18 组 command_failures 均为 0	最终 Release 日志
AI 结束条件	18 组均 rounds=3	最终 Release 日志

5.3 启动故障与运行验收

故障样本	预期行为	最终结果
缺 game.cfg	带路径诊断并退出	退出码 1
缺地图文件	带路径诊断并退出	退出码 1
max_rounds=0	范围错误并退出	退出码 1
缺中文字体	列出候选路径并退出	退出码 1
缺 SFML DLL	由 Windows 报告依赖缺失	0xC0000135
正常窗口	创建 AutoChess 窗口并正常关闭	退出码 0
暂停 1.5 秒	画面与核心状态保持不变	前后截图 SHA-256 相同
继续	核心恢复推进	继续后截图 SHA-256 改变

5.4 独立发布目录

dist 目录精确包含 10 个文件：AutoChessGame.exe、3 个 SFML Release DLL、game.cfg、units.cfg、skills.cfg、factions.cfg 和 2 张地图。6 个数据文件与源码 data 的 SHA-256 完全一致。项目外副本以 D:\ 作为工作目录仍能创建窗口并正常退出；删除外部副本的 game.cfg 后，错误路径明确指向外部 EXE 旁 data，证明没有读取 D:\1\data。

6 运行界面与结果

以下截图均来自第 10 天 Release x64 正式验收。原始 13 张截图中两组画面哈希重复：返回主菜单与初始主菜单相同，暂停延时与暂停画面相同，因此报告保留 11 张唯一画面。



图 6-1 主菜单：开始游戏、帮助与退出

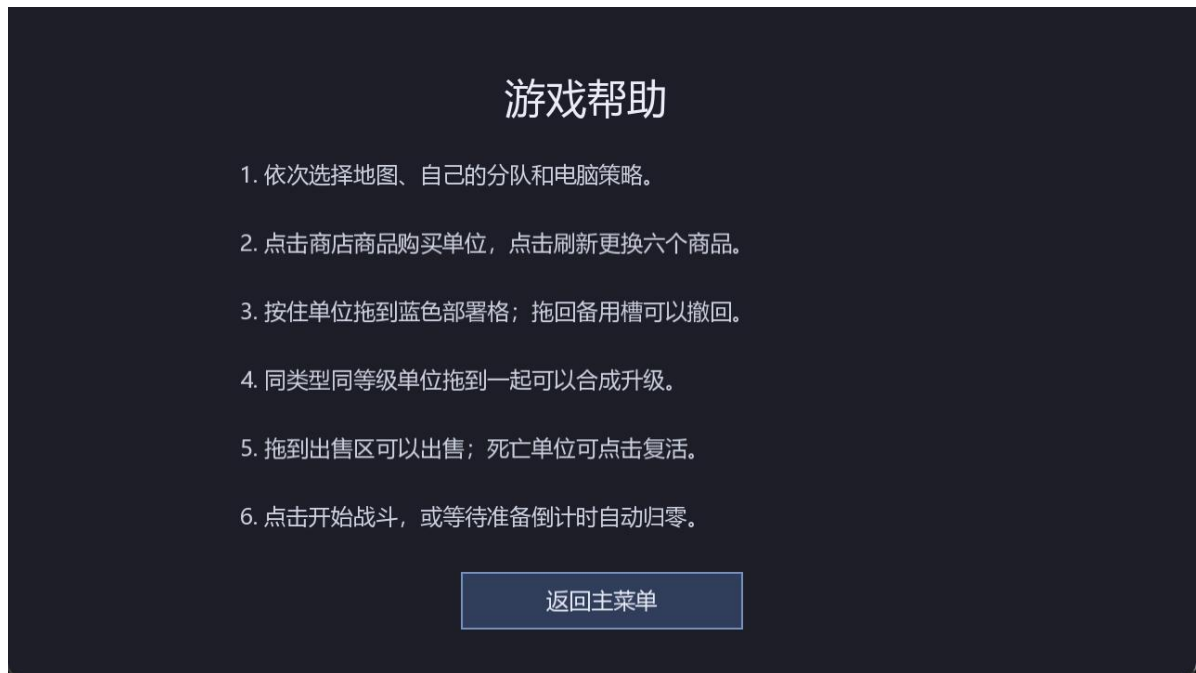


图 6-2 帮助页：新手操作说明



图 6-3 地图选择



图 6-4 玩家分队选择



图 6-5 AI 策略选择

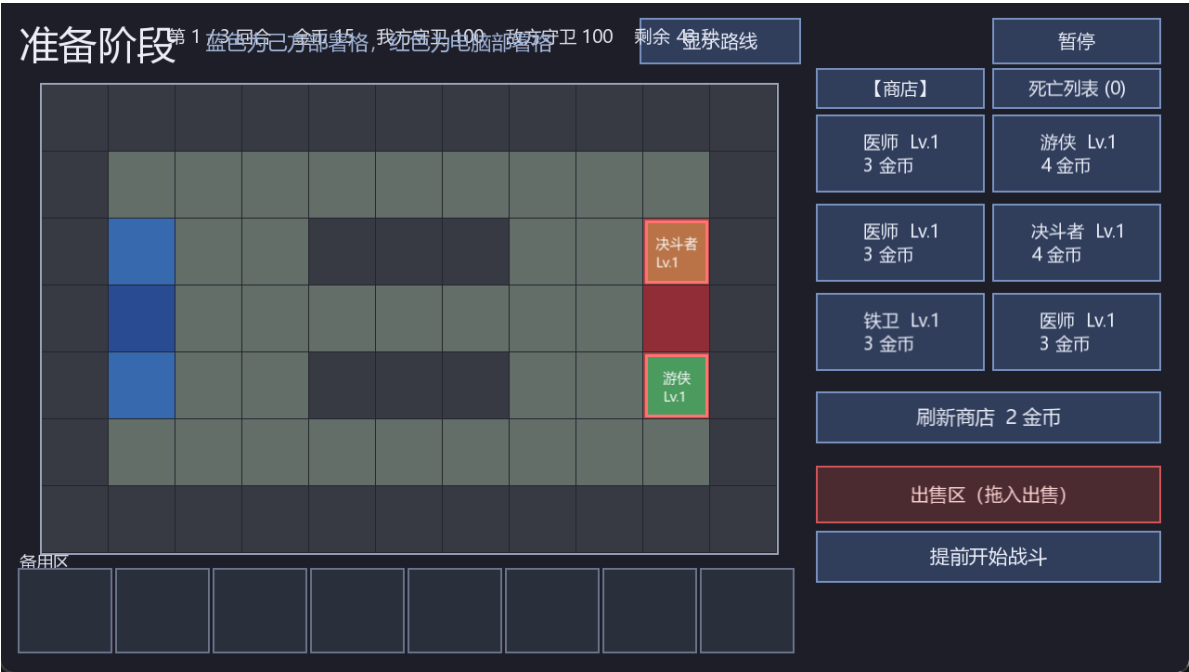


图 6-6 准备阶段完整 HUD 与商店



图 6-7 购买单位后备用区状态



图 6-8 拖拽部署后的棋盘状态

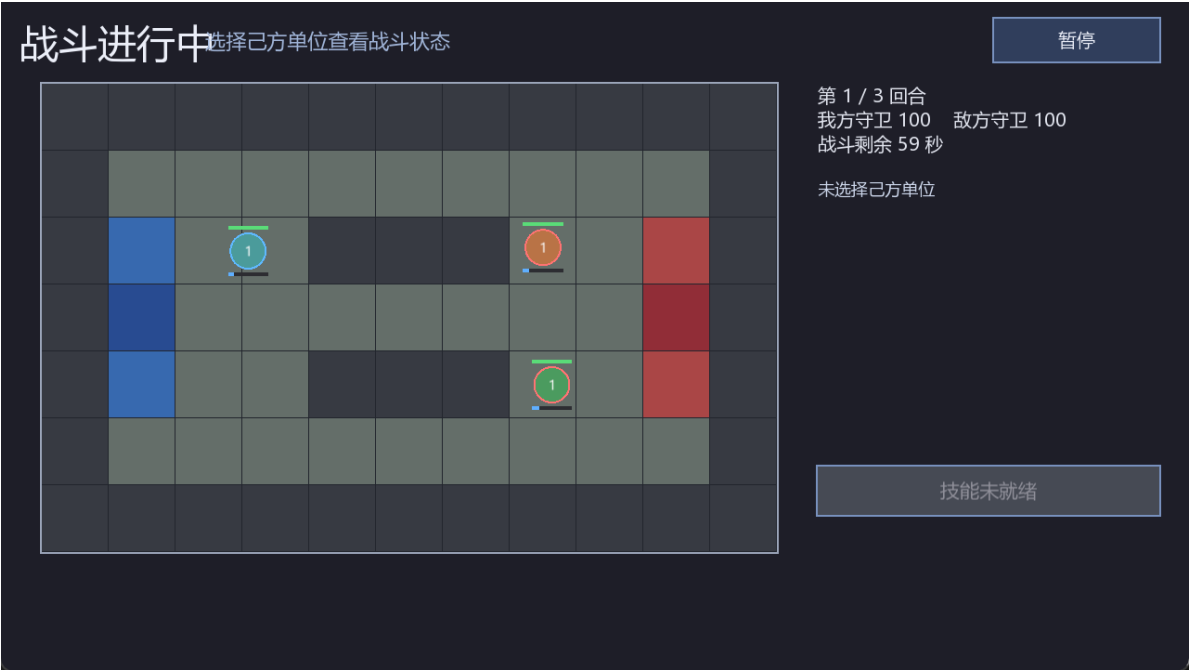


图 6-9 战斗单位、状态条与目标连线



图 6-10 暂停覆盖层



图 6-11 继续后恢复战斗推进

7 系统调试

7.1 CMake 工程文件名错误

现象：初始工程使用 CMakeList.txt，CMake 无法按约定发现根构建脚本。原因：标准文件名必须为 CMakeLists.txt，少了结尾字母 s。修复：将文件精确改名并重新配置。验证：随后建立 AutoChessCore、AutoChessGame、AutoChessTests 三个目标并通过第一天验收。

7.2 发布包数据路径依赖开发目录

现象：程序在构建目录可运行，但复制 EXE 与 DLL 后无法可靠独立读取旁置数据，存在依赖编译期源码路径的风险。原因：GameApp 直接使用 AUTOCHESSE_DATA_DIR，当前工作目录和 EXE 目录没有明确优先级。修复：新增 RuntimePaths，使用 GetModuleFileNameW 取得 EXE 目录；旁置 data 存在时固定使用它。验证：项目外启动退出码 0；临时移除外部 game.cfg 后，诊断仍指向外部 data，没有回退源码目录。

7.3 同帧索敌并列规则缺少直接断言

现象：实现已覆盖首次进入攻击范围的顺序，但测试没有直接证明两个目标在同一帧进入范围时选择较小单位编号。原因：原用例只构造了不同进入帧，无法覆盖并列分支。修复：增加同帧、同距

离、不同编号的最小回归用例，不修改生产规则。验证：新增断言通过，完整 Release 回归与 18 组 AI 输出保持一致。

7.4 调试方法总结

- 编译问题先缩小到 CMake 配置、编译、链接或 DLL 启动四个阶段，再保留完整日志。
- 规则问题使用最小值对象和固定种子构造单元测试，避免依赖 GUI 手工观察。
- 运行问题复制到带所有权标记的临时目录注入故障，绝不直接删除正式 data 或 dist 文件。
- 每次修复后先运行针对性测试，再运行完整 Release 测试和 `git diff --check`。

8 使用说明与编程体会

8.1 发布程序使用步骤

7. 打开 dist 目录，确认 AutoChessGame.exe、3 个 SFML DLL 和 data 目录位于同一级。
8. 双击 AutoChessGame.exe；若资源完整，将出现标题为 AutoChess 的主菜单窗口。
9. 首次使用先点击“帮助”，阅读购买、拖拽、合成、出售、复活和提前开战说明，再返回主菜单。
10. 点击“开始游戏”，依次选择地图、玩家分队和 AI 策略。
11. 准备阶段点击商店卡购买；按住己方单位拖到合法部署格，或拖回备用区/出售区。
12. 需要时点击刷新、合成或复活；点击提前开战，或等待 45 秒准备倒计时结束。
13. 战斗中点击己方单位，再点击技能按钮释放已就绪主动技能；可暂停和继续。
14. 回合结算后继续下一回合；最多 3 回合后在结果页选择重新开始、再来一局或返回菜单。

8.2 构建与测试步骤

15. 在 Visual Studio x64 Developer Command Prompt 中打开项目根目录。
16. 执行 `cmake --preset msvc-release --fresh`，确认找到现有 SFML 2.6.2。
17. 执行 `cmake --build --preset msvc-release`，确认 AutoChessGame 和 AutoChessTests 生成。
18. 运行 `build/msvc-release/AutoChessTests.exe`，确认最终汇总为 402 PASS、0 FAIL。
19. 执行 `cmake --build --preset msvc-release --target AutoChessDist`，重新组装 dist。

8.3 编程体会

本项目最大的收获是认识到“可运行”不等于“可交付”。如果界面直接修改规则、AI 读取内部对象、发布包依赖源码路径，即使演示一次成功，也很难稳定测试和复现。通过核心层隔离、统一命令、只读快照、固定步长和独立发布目录，复杂功能可以按小步骤完成，每一步都有清晰验收条件。

第二个收获是先冻结规则再写界面。配置格式、战斗事件顺序、经济比例和地图路线在早期确定后，后续 UI 和 AI 只需调用稳定接口。402 项自动断言与 18 组 AI 矩阵使最后一天主要处理发布和文档，而不是同时修复大量功能。

9 总结与展望

本项目已经完成任务书要求的自走棋购买、升级、排列和对战流程，并实现清晰类层次、抽象类、多重继承、虚函数多态、运算符重载、文件数据读写、策略型 AI、独立测试和 Release 发布。最终系统能够从主菜单稳定完成默认 3 回合，缺失资源时给出明确诊断。

后续扩展应保持现有边界：可增加精细贴图、动作动画、音效、更多单位/地图、动态寻路或网络控制器，但不得让 SFML 进入核心层，也不得绕过 GameCommand 修改 Match。任何新规则都应先增加配置字段和自动测试，再接入界面。

附录 A 关键源程序与源程序清单

关键源程序已在第 4 章列出并保留用途注释。本节给出仓库中的全部 C++ 源文件和测试文件清单，实际提交以 Git 仓库内容为准。

序号	文件	职责
1	src/core/Core.cpp	核心公共入口
2	src/core/Core.hpp	核心公共入口
3	src/core/ai/AiController.cpp	AI 接口、决策工具、策略或控制器
4	src/core/ai/AiController.hpp	AI 接口、决策工具、策略或控制器
5	src/core/ai/AiDecisionSupport.cpp	AI 接口、决策工具、策略或控制器
6	src/core/ai/AiDecisionSupport.hpp	AI 接口、决策工具、策略或控制器
7	src/core/ai/DefensiveStrategy.cpp	AI 接口、决策工具、策略或控制器
8	src/core/ai/DefensiveStrategy.hpp	AI 接口、决策工具、策略或控制器
9	src/core/ai/IAiStrategy.hpp	AI 接口、决策工具、策略或控制器
10	src/core/ai/OffensiveStrategy.cpp	AI 接口、决策工具、策略或控制器
11	src/core/ai/OffensiveStrategy.hpp	AI 接口、决策工具、策略或控制器
12	src/core/ai/RouteStrategy.cpp	AI 接口、决策工具、策略或控制器
13	src/core/ai/RouteStrategy.hpp	AI 接口、决策工具、策略或控制器
14	src/core/combat/BattleSetupService.cpp	固定步长战斗、伤害、索敌或战斗数据
15	src/core/combat/BattleSetupService.hpp	固定步长战斗、伤害、索敌或战斗数据
16	src/core/combat/BattleSimulation.cpp	固定步长战斗、伤害、索敌或战斗数据
17	src/core/combat/BattleSimulation.hpp	固定步长战斗、伤害、索敌或战斗数据
18	src/core/combat/BattleStatResolver.cpp	固定步长战斗、伤害、索敌或战斗数据
19	src/core/combat/BattleStatResolver.hpp	固定步长战斗、伤害、索敌或战斗数据
20	src/core/combat/BattleTypes.hpp	固定步长战斗、伤害、索敌或战斗数据
21	src/core/combat/CombatRules.cpp	固定步长战斗、伤害、索敌或战斗数据
22	src/core/combat/CombatRules.hpp	固定步长战斗、伤害、索敌或战斗数据
23	src/core/combat/TargetSelector.cpp	固定步长战斗、伤害、索敌或战斗数据
24	src/core/combat/TargetSelector.hpp	固定步长战斗、伤害、索敌或战斗数据
25	src/core/config/ConfigBundleLoader.cpp	配置解析、类型读取、文件加载或校验
26	src/core/config/ConfigBundleLoader.hpp	配置解析、类型读取、文件加载或校验
27	src/core/config/ConfigError.hpp	配置解析、类型读取、文件加载或校验
28	src/core/config/ConfigParser.cpp	配置解析、类型读取、文件加载或校验
29	src/core/config/ConfigParser.hpp	配置解析、类型读取、文件加载或校验
30	src/core/config/ConfigValueReader.cpp	配置解析、类型读取、文件加载或校验
31	src/core/config/ConfigValueReader.hpp	配置解析、类型读取、文件加载或校验
32	src/core/config/DefinitionConfigLoader.cpp	配置解析、类型读取、文件加载或校验
33	src/core/config/DefinitionConfigLoader.hpp	配置解析、类型读取、文件加载或校验
34	src/core/config/GameConfigLoader.cpp	配置解析、类型读取、文件加载或校验
35	src/core/config/GameConfigLoader.hpp	配置解析、类型读取、文件加载或校验
36	src/core/config/MapConfigLoader.cpp	配置解析、类型读取、文件加载或校验
37	src/core/config/MapConfigLoader.hpp	配置解析、类型读取、文件加载或校验
38	src/core/controllers/IPlayerController.hpp	核心公共入口
39	src/core/economy/DeploymentService.cpp	商店、价格、部署、合成、出售或复活
40	src/core/economy/DeploymentService.hpp	商店、价格、部署、合成、出售或复活
41	src/core/economy/EconomyTypes.hpp	商店、价格、部署、合成、出售或复活
42	src/core/economy/MergeService.cpp	商店、价格、部署、合成、出售或复活
43	src/core/economy/MergeService.hpp	商店、价格、部署、合成、出售或复活
44	src/core/economy/PriceRules.cpp	商店、价格、部署、合成、出售或复活
45	src/core/economy/PriceRules.hpp	商店、价格、部署、合成、出售或复活
46	src/core/economy/RosterService.cpp	商店、价格、部署、合成、出售或复活
47	src/core/economy/RosterService.hpp	商店、价格、部署、合成、出售或复活
48	src/core/economy/ShopService.cpp	商店、价格、部署、合成、出售或复活
49	src/core/economy/ShopService.hpp	商店、价格、部署、合成、出售或复活
50	src/core/map/MapTypes.hpp	地图、路线和格子数据
51	src/core/match/GameCommand.hpp	统一命令、对局阶段、回合结算或只读视图
52	src/core/match/Match.cpp	统一命令、对局阶段、回合结算或只读视图
53	src/core/match/Match.hpp	统一命令、对局阶段、回合结算或只读视图
54	src/core/match/MatchTypes.cpp	统一命令、对局阶段、回合结算或只读视图
55	src/core/match/MatchTypes.hpp	统一命令、对局阶段、回合结算或只读视图
56	src/core/match/ReadOnlyGameView.hpp	统一命令、对局阶段、回合结算或只读视图
57	src/core/match/RoundController.cpp	统一命令、对局阶段、回合结算或只读视图
58	src/core/match/RoundController.hpp	统一命令、对局阶段、回合结算或只读视图
59	src/core/model/Definitions.hpp	单位/玩家/配置基础模型
60	src/core/model/PlayerStateService.cpp	单位/玩家/配置基础模型
61	src/core/model/PlayerStateService.hpp	单位/玩家/配置基础模型
62	src/core/model/PlayerTypes.hpp	单位/玩家/配置基础模型
63	src/core/skills/Skill.cpp	技能抽象与技能效果系统

自走棋对战系统课程设计报告

64	src/core/skills/Skill.hpp	技能抽象与技能效果系统
65	src/core/skills/SkillSystem.cpp	技能抽象与技能效果系统
66	src/core/skills/SkillSystem.hpp	技能抽象与技能效果系统
67	src/core/units/Unit.cpp	单位抽象、角色与多重继承
68	src/core/units/Unit.hpp	单位抽象、角色与多重继承
69	src/game/GameApp.cpp	SFML 应用入口、主循环或运行路径
70	src/game/GameApp.hpp	SFML 应用入口、主循环或运行路径
71	src/game/RuntimePaths.cpp	SFML 应用入口、主循环或运行路径
72	src/game/RuntimePaths.hpp	SFML 应用入口、主循环或运行路径
73	src/game/controllers/BootstrapComputerController.cpp	真人或启动脚本控制器
74	src/game/controllers/BootstrapComputerController.hpp	真人或启动脚本控制器
75	src/game/controllers/HumanController.cpp	真人或启动脚本控制器
76	src/game/controllers/HumanController.hpp	真人或启动脚本控制器
77	src/game/main.cpp	SFML 应用入口、主循环或运行路径
78	src/game/rendering/BoardTransform.cpp	坐标换算、地图或单位绘制
79	src/game/rendering/BoardTransform.hpp	坐标换算、地图或单位绘制
80	src/game/rendering/MapRenderer.cpp	坐标换算、地图或单位绘制
81	src/game/rendering/MapRenderer.hpp	坐标换算、地图或单位绘制
82	src/game/rendering/UnitRenderer.cpp	坐标换算、地图或单位绘制
83	src/game/rendering/UnitRenderer.hpp	坐标换算、地图或单位绘制
84	src/game/screens/HelpScreen.cpp	菜单、帮助或对局页面
85	src/game/screens/HelpScreen.hpp	菜单、帮助或对局页面
86	src/game/screens/MainMenuScreen.cpp	菜单、帮助或对局页面
87	src/game/screens/MainMenuScreen.hpp	菜单、帮助或对局页面
88	src/game/screens/MatchScreen.cpp	菜单、帮助或对局页面
89	src/game/screens/MatchScreen.hpp	菜单、帮助或对局页面
90	src/game/screens/Screen.hpp	菜单、帮助或对局页面
91	src/game/ui/Button.cpp	按钮和界面动作
92	src/game/ui/Button.hpp	按钮和界面动作
93	src/game/ui/UiAction.hpp	按钮和界面动作
94	tests/AiIntegrationTests.cpp	自动测试入口、断言或集成测试
95	tests/AiIntegrationTests.hpp	自动测试入口、断言或集成测试
96	tests/AiTests.cpp	自动测试入口、断言或集成测试
97	tests/AiTests.hpp	自动测试入口、断言或集成测试
98	tests/MatchAcceptanceTests.cpp	自动测试入口、断言或集成测试
99	tests/MatchAcceptanceTests.hpp	自动测试入口、断言或集成测试
100	tests/MatchScriptedTests.cpp	自动测试入口、断言或集成测试
101	tests/MatchScriptedTests.hpp	自动测试入口、断言或集成测试
102	tests/RuntimePathsTests.cpp	自动测试入口、断言或集成测试
103	tests/RuntimePathsTests.hpp	自动测试入口、断言或集成测试
104	tests/TestMain.cpp	自动测试入口、断言或集成测试

附录 B 完整课程设计任务书与评分表

以下 4 页由原文件《2026 程设 2 大作业任务书-自走棋对战系统.docx》直接渲染并整页嵌入。
评分表位于本报告源程序清单之后，符合任务书位置要求。

课程设计报告

课程名称 计算机程序设计基础2

班 级 _____
学 号 _____
姓 名 _____

2026 年 5 月 19 日

附图 B-1 原课程设计任务书第 1 页

一、设计内容与设计要求

1. 课程设计目的

面向对象程序设计课程是集中实践性环节之一，是学习完《计算机程序设计基础2》C++面向对象程序设计课程后进行的一次全面的综合练习。要求学生达到熟练掌握C++语言的基本知识和技能；基本掌握面向对象程序设计的思想和方法；能够利用所学的基本知识和技能，解决简单的面向对象程序设计问题，从而提高动手编程解决实际问题的能力。**尤其重视创新思维培养。**

2. 课题题目

1) 自走棋对战系统：自走棋是一种策略游戏，玩家需要购买、升级、排列棋子以击败对手。

3. 课题内容（仅供参考）

- 系统需求分析：分析自走棋对战系统的功能需求，包括棋子购买、升级、排列、对战等。
- 系统设计：设计自走棋对战系统的类层次结构，确定基类和派生类，以及类的成员函数和成员变量。
- 系统实现：
 - 1) 实现棋子类，包括棋子的名称、攻击力、血量等属性，以及攻击、移动等行为。
 - 2) 实现玩家类，包括玩家的名称、血量、金币、棋子等属性，以及购买、升级、排列棋子等行为。
 - 3) 实现战场类，包括战场的地图、棋子位置等属性，以及棋子的移动、攻击等行为。
 - 4) 实现系统的菜单功能，包括开始游戏、购买棋子、升级棋子、排列棋子、开始对战等。
- 设计策略型AI玩家互相对战。（加分项）

4. 文档设计要求

4.1 每个同学都单独完成1道课题。后面有范题，仅供同学们参考，不列入本次课程设计的课题。

4.2 对于程设题目，按照范题的格式。自行虚构软件需求。并按照第4点要求，编写设计文档。基本要求系统中设计的类的数目不少于4个，每个类中要有各自的属性（多于3个）和方法（多于3个）；需要定义一个抽象类，采用继承方式派生这些类。并设计一个多重继承的派生类。在程序设计中，引入虚函数的多态性、运算符重载等机制。

5. 程序设计的基本要求:

- (1) 要求利用面向对象的方法以及C++的编程思想来完成系统的设计;
- (2) 要求在设计的过程中, 建立清晰的类层次;
- (3) 根据课题完成以下主要工作: ①完成系统需求分析: 包括系统设计目的与意义; 系统功能需求(系统流程图); 输入输出的要求。②完成系统总体设计: 包括系统功能分析; 系统功能模块划分与设计(系统功能模块图)。③完成系统详细设计: 数据文件; 类层次图; 界面设计与各功能模块实现。④系统调试: 调试出现的主要问题, 编译语法错误及修改, 重点是运行逻辑问题修改和调整。⑤使用说明书及编程体会: 说明如何使用你编写的程序, 详细列出每一步的操作步骤。⑥关键源程序(带注释)。
- (4) 自己设计测试数据, 将测试数据存在文件中, 通过文件进行数据读写来获得测试结果。
- (5) 按规定格式完成课程设计报告, 并在网络学堂上按时提交。
- (6) 不得抄袭他人程序、课程设计报告, 每个人应独立完成, 在程序和设计报告中体现自己的个性设计。

5、进度安排

小学期 第3周			

注: 1、一定要保留自己那个课题的完整任务书在课程设计报告里面。

2、“评分表”放在“附录: 源程序清单”的后面。

附录1：评分表

项目	评价	
设计方案的合理性与创新性	3	
设计与调试结果	4	
设计说明书的质量	1	
程序基本要求涵盖情况	4	
程序代码编写素养情况	2	
课程设计周表现情况	1	
综合成绩	15	